

Secret

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Descrivere i tipi di Secret Kubernetes ([Opaque](#), [kubernetes.io/tls](#), [kubernetes.io/dockerconfigjson](#), [kubernetes.io/service-account-token](#)) e i relativi use case
- Creare Secret in modo sicuro tramite `oc create secret` evitando di esporre dati sensibili nella shell history
- Consumare Secret come variabili d'ambiente e come file montati con permessi restrittivi
- Applicare le best practice per evitare il commit di Secret in Git (Sealed Secrets, External Secrets Operator)
- Eseguire la rotazione di un Secret e forzare il rollout del Deployment
- Configurare l'integrazione con Azure Key Vault tramite Secrets Store CSI Driver su ARO

Teoria e Concetti Chiave

Cos'è un Secret

Un **Secret** è un oggetto Kubernetes namespace-scoped che archivia dati sensibili (password, token, certificati TLS, chiavi API) in formato Base64. A differenza dei ConfigMap, i Secret sono:

- Trasmessi ai nodi **solo se un Pod nel nodo li richiede** (non distribuiti in broadcast)
- Archiviati in **etcd** (cifrati at-rest se configurato [EncryptionConfiguration](#))
- Montati in **tmpfs** (memoria volatile, non su disco) nei container

⚠ Attenzione: I Secret Kubernetes sono codificati in Base64, **non cifrati**. Chiunque abbia accesso in lettura all'oggetto Secret (via `oc get secret`) può decodificarli. Su ARO, etcd è cifrato at-rest per impostazione predefinita.

Tipi di Secret

Tipo	Chiavi obbligatorie	Use case
Opaque	(nessuna)	Dati generici: password, API key, connection string
kubernetes.io/tls	tls.crt , tls.key	Certificati TLS per Route/Ingress
kubernetes.io/dockerconfigjson	.dockerconfigjson	Credenziali per registry privati
kubernetes.io/service-account-token	token , ca.crt , namespace	Token di autenticazione ServiceAccount (auto-gestiti)
kubernetes.io/basic-auth	username , password	Autenticazione HTTP Basic
kubernetes.io/ssh-auth	ssh-privatekey	Chiavi SSH private

Creazione di Secret

Opaque da literal:

```
oc create secret generic db-credentials \  
  --from-literal=username=appuser \  
  --from-literal=password=S3cr3tP@ssw0rd \  
  -n myproject
```

TLS da file certificato:

```
oc create secret tls myapp-tls \  
  --cert=tls.crt \  
  --key=tls.key \  
  -n myproject
```

Docker registry:

```
oc create secret docker-registry registry-credentials \  
  --docker-server=registry.example.com \  
  --docker-username=user \  
  --docker-password=token \  
  --docker-email=user@example.com \  
  -n myproject
```

Consumo come Variabili d'Ambiente

```
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-credentials    # ← nome del Secret
      key: password          # ← chiave nel Secret
      optional: false        # ← Pod non si avvia se Secret assente
```

⚠ **Attenzione:** Come per i ConfigMap, le env vars da Secret vengono lette **una sola volta** all'avvio del container. Un cambio di Secret richiede riavvio del Pod. Inoltre, le env vars sono visibili in [oc describe pod](#) (non nel valore, ma nel nome), nei log di sistema e nei tool di debug.

Consumo come Volume (Più Sicuro)

```
volumeMounts:
- name: db-secret-vol
  mountPath: /secrets/db    # ← directory nel container
  readOnly: true           # ← sola lettura obbligatoria
volumes:
- name: db-secret-vol
  secret:
    secretName: db-credentials
    defaultMode: 0400      # ← permessi restrittivi (solo owner può leggere)
    items:
      - key: password
        path: db-password  # ← file /secrets/db/db-password
```

💡 **Suggerimento:** Preferire il montaggio come file rispetto alle env vars per Secret usati da applicazioni Java/Spring: il file può essere ricaricato senza riavvio del Pod se il framework lo supporta (es. Spring Cloud Kubernetes).

Rotazione di un Secret

Il processo raccomandato per ruotare un Secret senza downtime:

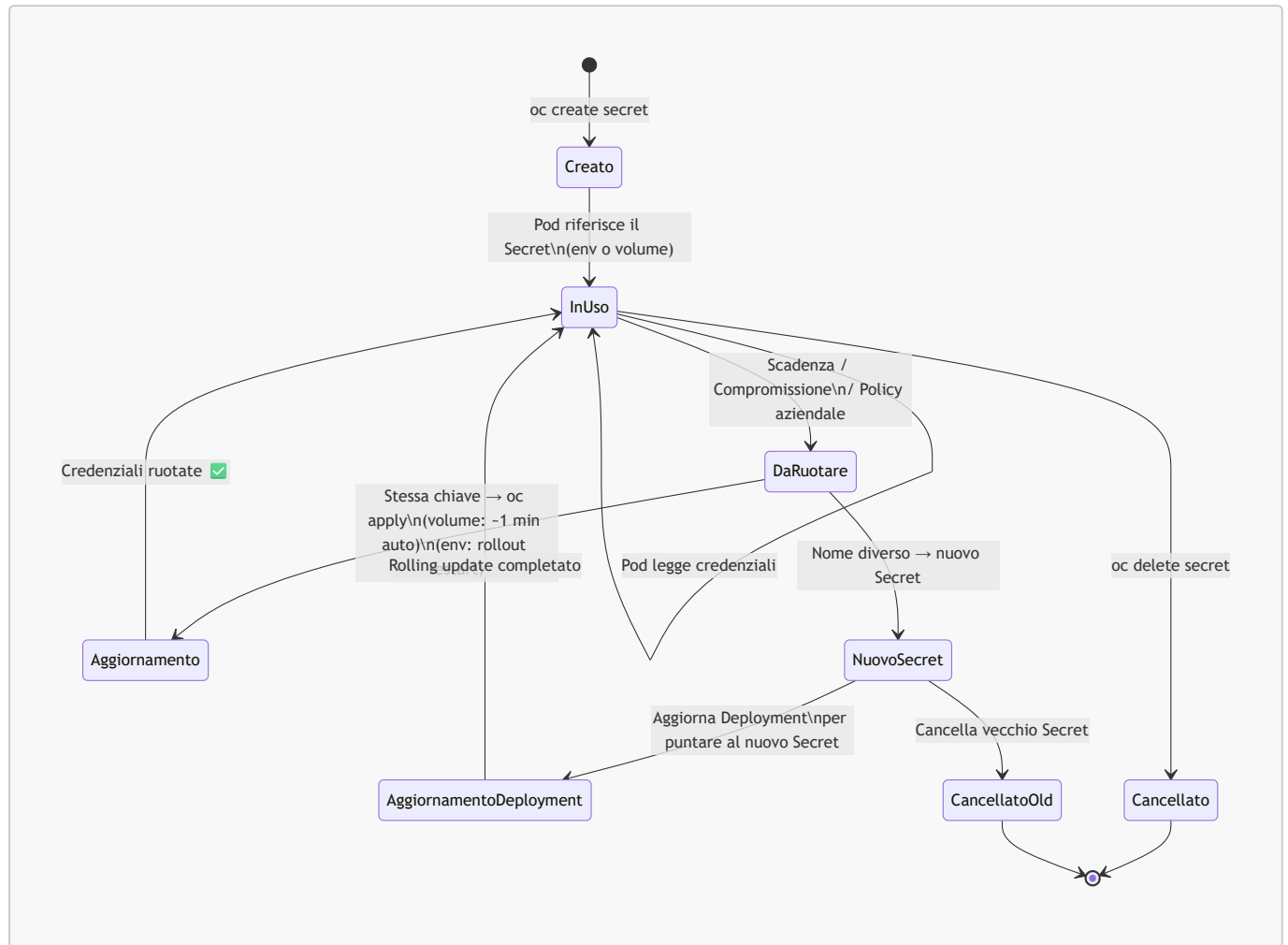
1. Creare il nuovo Secret con credenziali aggiornate (nome diverso o stesso nome)
2. Se stesso nome → oc apply aggiorna il Secret → volume mounts aggiornati in ~1 min
3. Se nome diverso → aggiornare il Deployment per puntare al nuovo Secret → rollout
4. Verificare che i Pod usino le nuove credenziali
5. Cancellare il vecchio Secret se non più necessario

Best Practice: Evitare Commit in Git

Approccio	Sicurezza	Complessità	Use case
Secret Kubernetes nativi	Media (Base64)	Bassa	Ambienti non critici, demo
Sealed Secrets (Bitnami)	Alta (cifrati con chiave cluster)	Media	GitOps, commit sicuro in Git
External Secrets Operator	Molto alta (backend esterno)	Media-Alta	Produzione con Azure Key Vault, AWS Secrets Manager
Secrets Store CSI Driver	Molto alta (montaggio diretto da AKV)	Media	ARO con Azure Key Vault

Diagrammi

Ciclo di Vita e Rotazione di un Secret



Comandi Pratici con Output Atteso

```
# Creare un Secret Opaque con credenziali database
oc create secret generic db-credentials \
  --from-literal=username=appuser \
  --from-literal=password='S3cr3t!2026' \
  -n myproject

# Output atteso:
# secret/db-credentials created
```

```
# Visualizzare il Secret (dati in Base64)
oc get secret db-credentials -n myproject -o yaml

# Output atteso (estratto):
# apiVersion: v1
# data:
#   password: UzNjcjN0ITlWmJY=    ← Base64 di 'S3cr3t!2026'
#   username: YXBwdXNlcg==        ← Base64 di 'appuser'
# kind: Secret
# type: Opaque
```

```
# Decodificare un valore Base64 (solo per verifica in ambiente sicuro)
oc get secret db-credentials -n myproject \
  -o jsonpath='{.data.password}' | base64 -d

# Output atteso:
# S3cr3t!2026
```

```
# Creare un Secret TLS da file certificato
oc create secret tls myapp-tls \
  --cert=certs/tls.crt \
  --key=certs/tls.key \
  -n myproject

# Output atteso:
# secret/myapp-tls created
```

```
# Creare un Secret per un registry privato (Azure Container Registry)
oc create secret docker-registry acr-credentials \
  --docker-server=myacr.azurecr.io \
  --docker-username=myacr \
```

```
--docker-password=$(az acr credential show --name myacr --query
passwords[0].value -o tsv) \
-n myproject
```

```
# Output atteso:
# secret/acr-credentials created
```

```
# Associare il Secret al ServiceAccount default per pull automatico
oc secrets link default acr-credentials --for=pull -n myproject
```

```
# Output atteso:
# (nessun output → successo)
# Verificare:
oc describe serviceaccount default -n myproject | grep "Image pull secrets"
# Image pull secrets: default-dockercfg-..., acr-credentials
```

```
# Descrivere un Secret senza mostrare i dati
oc describe secret db-credentials -n myproject
```

```
# Output atteso:
# Name:          db-credentials
# Namespace:     myproject
# Type:          Opaque
# Data
# ====
# password:  12 bytes
# username:  7 bytes
```

```
# Aggiornare un Secret esistente (rotazione password)
oc create secret generic db-credentials \
  --from-literal=username=appuser \
  --from-literal=password='N3wP@ssw0rd2026' \
  --dry-run=client -o yaml | oc apply -f - -n myproject
```

```
# Output atteso:
# secret/db-credentials configured
```

```
# Forzare rollout per recepire la nuova password nelle env vars
oc rollout restart deployment/myapp -n myproject
```

```
# Output atteso:
# deployment.apps/myapp restarted
```

Esempi YAML Completì

Secret Opaque + Secret TLS + Pod che li Consuma

```
# Secret Opaque: credenziali database
# NOTA: i valori data devono essere in Base64 (echo -n 'value' | base64)
# In produzione: generare con oc create secret, non scrivere a mano
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials          # ← nome del Secret
  namespace: myproject
  labels:
    app: myapp
    tier: database
type: Opaque                   # ← tipo generico per dati non
strutturati
data:
  # echo -n 'appuser' | base64 → YXBwdXNlcg==
  username: YXBwdXNlcg==       # ← Base64("appuser")
  # echo -n 'S3cr3t!2026' | base64 → UzNjcjN0ITlwMjY=
  password: UzNjcjN0ITlwMjY=  # ← Base64("S3cr3t!2026")
  # echo -n 'jdbc:postgresql://postgres-svc:5432/appdb' | base64
  connection-string: amRiYzpwY3N0Z3Jlc3FsOi8vcG9zdGdyZXMtY3ZjOjU0MzIvYXBwZGI=
---
# Secret TLS: certificato per Route HTTPS
apiVersion: v1
kind: Secret
metadata:
  name: myapp-tls              # ← nome del Secret TLS
  namespace: myproject
type: kubernetes.io/tls       # ← tipo specifico TLS
data:
  # Contenuto del certificato PEM in Base64
  tls.crt: LS0tLS1CRUdJT...    # ← base64 del file tls.crt (troncato per
esempio)
  tls.key: LS0tLS1CRUdJT...    # ← base64 del file tls.key (troncato per
esempio)
---
# Pod che consuma il Secret Opaque sia come env var che come volume
apiVersion: v1
kind: Pod
metadata:
  name: myapp-with-secrets
  namespace: myproject
spec:
  containers:
    - name: app
      image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
      command: ["/bin/sh", "-c", "echo DB_USER=$DB_USERNAME && cat /secrets/conn &&
sleep 3600"]
```



```

resources:
  requests:
    memory: "128Mi"
    cpu: "100m"
  limits:
    memory: "256Mi"
    cpu: "500m"
env:
# Singola chiave come env var (metodo meno sicuro ma compatibile)
- name: DB_USERNAME
  valueFrom:
    secretKeyRef:
      name: db-credentials          # ← nome del Secret
      key: username                 # ← chiave specifica
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: password
volumeMounts:
- name: db-secret-vol
  mountPath: /secrets              # ← directory di mount (tmpfs)
  readOnly: true                  # ← sola lettura obbligatoria
volumes:
- name: db-secret-vol
  secret:
    secretName: db-credentials     # ← Secret da montare
    defaultMode: 0400             # ← permessi: solo owner (UID del
processo)
    items:
      - key: connection-string
        path: conn                 # ← file /secrets/conn con la connection
string

```

Secret dockerconfigjson per Azure Container Registry

```

# Secret per autenticazione a Azure Container Registry (ACR)
# In produzione: creare con 'oc create secret docker-registry'
apiVersion: v1
kind: Secret
metadata:
  name: acr-credentials            # ← nome del Secret registry
  namespace: myproject
  annotations:
    # Documentare l'origine del Secret
    secret.source: "Azure Container Registry - myacr.azurecr.io"
    secret.rotation-policy: "90 days"
type: kubernetes.io/dockerconfigjson # ← tipo specifico per registry
data:
  # Valore generato con:

```

```
# oc create secret docker-registry acr-credentials \
#   --docker-server=myacr.azurecr.io \
#   --docker-username=myacr \
#   --docker-password=<token> \
#   --dry-run=client -o jsonpath='{.data.\.dockerconfigjson}'
.dockerconfigjson: eyJhdXRocyI6eyJteWFjci5henVyZWNYLmlvIjp7ImF1dGgiOiJiWGxo...
---
```

Deployment che usa il Secret per fare pull da ACR

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  namespace: myproject
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      imagePullSecrets:
        - name: acr-credentials          # ← Secret per pull da registry privato
      containers:
        - name: app
          image: myacr.azurecr.io/myapp:1.0.0 # ← immagine su ACR privato
          resources:
            requests:
              memory: "256Mi"
              cpu: "200m"
            limits:
              memory: "512Mi"
              cpu: "1000m"
```

Note Specifiche per Azure ARO

etcd Cifrato at-rest su ARO

Su ARO, etcd è cifrato at-rest per impostazione predefinita usando AES-CBC. I Secret sono quindi protetti anche a livello di storage, a differenza di cluster Kubernetes vanilla non configurati.

```
# Verificare lo stato della cifratura etcd su ARO
oc get apiserver cluster -o jsonpath='{.spec.encryption.type}'
# Output: aescbc (oppure aesgcm su versioni recenti)
```

Integrazione Azure Key Vault con Secrets Store CSI Driver

Su ARO, il pattern raccomandato per ambienti di produzione è usare **Azure Key Vault** come sorgente di Secret, sincronizzandoli automaticamente tramite il **Secrets Store CSI Driver**:

```
# Installare il provider AKV per Secrets Store CSI Driver
oc apply -f https://raw.githubusercontent.com/Azure/secrets-store-csi-driver-provider-azure/master/deployment/provider-azure-installer.yaml

# Creare un SecretProviderClass per leggere da AKV
oc apply -f - <<EOF
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: azure-keyvault-secrets
  namespace: myproject
spec:
  provider: azure
  parameters:
    usePodIdentity: "false"
    clientId: "<managed-identity-client-id>" # ← Workload Identity
    keyvaultName: "mykeyvault"
    cloudName: "AzurePublicCloud"
    tenantId: "<azure-tenant-id>"
    objects: |
      array:
        - |
          objectName: db-password
          objectType: secret
          objectVersion: ""
  secretObjects:
    - secretName: db-credentials-from-akv # ← Secret K8s sincronizzato da AKV
      type: Opaque
      data:
        - objectName: db-password
          key: password
EOF
```

💡 **Suggerimento:** Usare **Workload Identity** (federazione OIDC) invece di Service Principal con client secret per autenticare il Secrets Store CSI Driver verso Azure Key Vault. Workload Identity non richiede rotazione manuale di credenziali.

Confronto Secret Nativi K8s vs Azure Key Vault-Synced

Caratteristica	Secret K8s nativi	AKV + Secrets Store CSI
Cifratura at-rest	etcd AES su ARO	Azure Key Vault HSM
Audit trail	Solo OpenShift audit log	Azure Monitor + AKV audit log
Rotazione automatica	Manuale	Automatica (AKV + re-sync)
Accesso cross-cluster	No (namespace-scoped)	Sì (centralized in AKV)
Complessità setup	Bassa	Media-Alta

Limitazione IAM per Secret su ARO

Su ARO, l'accesso ai Secret è governato da RBAC OpenShift. Per principio del minimo privilegio:

```
# Creare un Role che permette di leggere solo Secret specifici
oc create role secret-reader \
  --verb=get \
  --resource=secrets \
  --resource-name=db-credentials \
  -n myproject
```

Riepilogo dei Punti Chiave

- I Secret Kubernetes sono codificati in Base64 (non cifrati): su ARO etcd è cifrato at-rest per default; limitare l'accesso RBAC ai Secret in produzione
 - I tipi principali sono `Opaque` (generico), `kubernetes.io/tls` (certificati), `kubernetes.io/dockerconfigjson` (registry)
 - Il consumo come **volume con defaultMode: 0400** è più sicuro rispetto alle env vars perché i valori non appaiono nelle variabili di processo e supportano l'aggiornamento live
 - Non committare mai Secret in chiaro in Git: usare Sealed Secrets, External Secrets Operator o Azure Key Vault con Secrets Store CSI Driver
 - La rotazione di un Secret tramite `oc apply` aggiorna i volume mounts automaticamente (~1 min); le env vars richiedono `oc rollout restart`
 - Su ARO, il pattern raccomandato per produzione è Azure Key Vault + Secrets Store CSI Driver con Workload Identity
-

Domande di Verifica

1. Qual è la codifica usata per i valori nel campo `data` di un Secret Kubernetes?

- A) AES-256 con chiave del cluster
- B) Base64 ☒
- C) SHA-256 (hash non reversibile)
- D) UTF-8 in chiaro

2. Quale tipo di Secret si usa per fornire credenziali di pull da un registry privato (es. Azure Container Registry)?

- A) `kubernetes.io/tls`
- B) `kubernetes.io/service-account-token`
- C) `kubernetes.io/dockerconfigjson` ☒
- D) `Opaque`

3. Un Secret viene montato come volume in `/secrets` con `defaultMode: 0400`. Un container tenta di scrivere in quel path. Cosa accade?

- A) Il container scrive con successo perché è il processo principale
- B) Il container riceve un errore "Permission denied" ☒
- C) Il container scrive ma le modifiche non vengono persistite
- D) Il Pod viene riavviato automaticamente

4. Qual è il vantaggio principale di usare Azure Key Vault con Secrets Store CSI Driver rispetto ai Secret Kubernetes nativi su ARO?

- A) I Secret AKV hanno dimensioni illimitate
- B) I Secret AKV non richiedono RBAC per l'accesso
- C) AKV fornisce rotazione automatica, audit trail centralizzato e cifratura HSM ☒
- D) I Secret AKV sono aggiornati istantaneamente senza delay

5. Qual è la best practice per ruotare un Secret usato come variabile d'ambiente da un Deployment?

- A) Cancellare il Pod manualmente; Kubernetes crea automaticamente un Pod con il nuovo Secret
- B) Aggiornare il Secret con `oc apply`, poi eseguire `oc rollout restart deployment/myapp` ☒
- C) Eliminare e ricreare il Deployment da zero
- D) Il Deployment rileva automaticamente il cambio del Secret e fa rolling update